# INTERNET PROGRAMMING LAB

**Experiment No. 14**

---

**Title:** Online Second-hand Marketplace (ReNewMarket)

## AIM

To design and implement a web application for buying and selling second-hand goods using HTML, CSS and JavaScript, in which users can register, list items, search and filter, negotiate prices and place orders, and the administrator can manage users, categories, commodities and the logistics status of orders.

## ALGORITHM

**Step 1:** Start. Load the stored data (users, categories, items, offers, orders) from the browser storage; if it is empty, load the demo data.

**Step 2:** Display the Market page with all unsold items, a search box, a category filter and a price filter.

**Step 3:** Registration: read name, username, e-mail and password; validate the fields, reject a duplicate username and store the new user.

**Step 4:** Login: choose the role. For Administrator compare the credentials with admin/admin123; for a User match the username and password with the stored users; otherwise show an error.

**Step 5:** Search and filter: select the unsold items whose title contains the keyword and which match the chosen category and maximum price; redraw the item cards.

**Step 6:** Sell an item: a logged-in user enters title, category, price and description; validate and add the item to the item list.

**Step 7:** Buy Now: check that the user is logged in and is not the seller; mark the item as sold and create an order with logistics status "Pending".

**Step 8:** Negotiate: the buyer enters an offer amount, which is stored as a "Pending" offer. The seller opens My Transactions and accepts or rejects it; for an accepted offer the buyer completes the purchase at the agreed price.

**Step 9:** Administrator - User Management: create a user account by setting a login ID and password, edit the name or password, or delete a user.

**Step 10:** Administrator - Category Management: add a new unique category or delete an existing category.

**Step 11:** Administrator - Commodity Management: edit the title and price of an item or delete the item.

**Step 12:** Administrator - Order Management: change the logistics status of an order to Pending, Shipped or Received, or delete the order. After "Received" the buyer can post feedback.

**Step 13:** Save the data after every operation. Logout and Stop.

# PROGRAM

*File: p14.html (HTML, internal CSS and JavaScript in a single file)*

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ReNewMarket - Second-hand Marketplace</title>
<style>
  *{box-sizing:border-box;font-family:Segoe UI,Arial,sans-serif}
  body{margin:0;background:#eef6ee;color:#1f3a28}
  header{background:#2e7d32;color:#fff;padding:12px 22px;display:flex;justify-content:space-between;align-
      items:center;flex-wrap:wrap;gap:6px}
  header h2{margin:0}
  nav button{background:none;border:0;color:#fff;cursor:pointer;font-size:14px;padding:7px 11px;border-
      radius:4px}
  nav button.on,nav button:hover{background:#43a047}
  main{max-width:1000px;margin:18px auto;padding:0 14px}
  .card{background:#fff;border-radius:8px;padding:14px;margin-bottom:14px;box-shadow:0 1px 4px #0002}
  .bar{display:flex;gap:8px;flex-wrap:wrap;margin-bottom:10px}.bar>*{flex:1;min-width:130px}
  input,select,textarea{padding:8px;border:1px solid #b6cdb8;border-radius:5px;font-size:14px;width:100%}
  .btn{background:#2e7d32;color:#fff;border:0;padding:7px 12px;border-radius:5px;cursor:pointer;font-
      size:13px;width:auto}
  .btn.red{background:#c0392b}.btn.grey{background:#6b7c6e}.btn.blue{background:#1565c0}
  .grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:12px}
  .price{color:#c0392b;font-weight:700}.tag{display:inline-block;background:#dcefdc;border-
      radius:12px;padding:2px 9px;font-size:12px;margin:1px}
  table{width:100%;border-collapse:collapse}td,th{padding:7px;border-bottom:1px solid #dbe7dc;text-
      align:left;font-size:13px}
  .msg{padding:10px;border-radius:5px;margin-
      bottom:10px;background:#e2f6e9;color:#17653a}.err{background:#fde7e7;color:#a12626}
  .small{font-size:12px;color:#6b7c6e}.hidden{display:none}
</style>
</head>
<body>
<header><h2>&#9851; ReNewMarket</h2><nav id="nav"></nav></header>
<main><div id="msg"></div><div id="view"></div></main>

<script>
/* ---------- Data store (saved in browser localStorage) ---------- */
const load=(k,d)=>{try{return JSON.parse(localStorage.getItem(k))||d}catch(e){return d}};
const save=(k,v)=>{try{localStorage.setItem(k,JSON.stringify(v))}catch(e){}};
let db=load("rm_db",{
  users:[{id:1,name:"Ravi Kumar",user:"ravi",pass:"ravi123",email:"ravi@mail.com"},{id:2,name:"Divya
      S",user:"divya",pass:"divya123",email:"divya@mail.com"}],
  cats:["Electronics","Furniture","Clothing","Collectibles"],
  items:[
    {id:1,title:"Used iPhone 11 (64GB)",cat:"Electronics",price:21000,seller:"ravi",desc:"Good condition, with
        charger",sold:false},
    {id:2,title:"Wooden Study Table",cat:"Furniture",price:3500,seller:"divya",desc:"Solid teak, minor
        scratches",sold:false},
    {id:3,title:"Denim Jacket (M)",cat:"Clothing",price:800,seller:"ravi",desc:"Worn twice",sold:false},
    {id:4,title:"Vintage Film Camera",cat:"Collectibles",price:6500,seller:"divya",desc:"Fully working,
        1980s",sold:false}],
  offers:[],orders:[],fb:[],nid:{item:5,offer:1,order:100,user:3}});
const persist=()=>save("rm_db",db);
let me=null, role=null, page="market";
const $=id=>document.getElementById(id), inr=n=>"₹"+Number(n).toLocaleString("en-IN");
function note(t,b){ $("msg").innerHTML=`<div class="msg ${b?"err":""}">${t}</div>`;
    setTimeout(()=>$("msg").innerHTML="",4000); }

/* ---------- Navigation ---------- */
function go(p){
  page=p; persist();
  const menu = role==="admin" ? [["admin_users","User Management"],["admin_cats","Category
      Management"],["admin_items","Commodity Management"],["admin_orders","Order Management"]]
    : [["market","Market"],["sell","Sell an Item"],["tx","My Transactions"]];
  $("nav").innerHTML = menu.map(m=>`<button class="${m[0]===p?"on":""}"
      onclick="go('${m[0]}')">${m[1]}</button>`).join("")
    + (role?`<button onclick="logout()">Logout (${role==="admin"?"admin":me.user})</button>`:`<button
        class="${p==="login"?"on":""}" onclick="go('login')">Login / Register</button>`);
  $("view").innerHTML = views[p]();
}
const views={};

/* ---------- Login / Register ---------- */
views.login=()=>`<div class="bar"><div class="card"><h3>Login</h3><select id="lr"><option
    value="user">User</option><option value="admin">Administrator</option></select><br><br>
  <input id="lu" placeholder="Username"><br><br><input id="lp" type="password"
      placeholder="Password"><br><br><button class="btn" onclick="login()">Login</button>
  <p class="small">Admin demo: admin / admin123   User demo: ravi / ravi123</p></div>
  <div class="card"><h3>Register</h3><input id="rn" placeholder="Full name"><br><br><input id="ru"
      placeholder="Username"><br><br><input id="re" placeholder="E-mail"><br><br>
  <input id="rp" type="password" placeholder="Password"><br><br><button class="btn"
      onclick="register()">Register</button></div></div>`;
function login(){
  const r=$("lr").value,u=$("lu").value,p=$("lp").value;
  if(r==="admin"){ if(u==="admin"&&p==="admin123"){role="admin";me={user:"admin"};return go("admin_users");}
      return note("Invalid admin login.",true); }
  const x=db.users.find(a=>a.user===u&&a.pass===p); if(!x) return note("Invalid username or password.",true);
  me=x; role="user"; note("Welcome, "+x.name+"!"); go("market");
}
function register(){
```

```
  const n=$("rn").value.trim(),u=$("ru").value.trim(),e=$("re").value,p=$("rp").value;
  if(!n||!u||!/^\S+@\S+\.\S+$/.test(e)||p.length<4) return note("Enter valid name, username, e-mail and password
       (min 4).",true);
  if(db.users.some(a=>a.user===u)) return note("Username already exists.",true);
  db.users.push({id:db.nid.user++,name:n,user:u,pass:p,email:e}); persist(); note("Registered successfully.
      Please login.");
}
function logout(){ me=null; role=null; go("market"); }

/* ---------- Market: search, filter, offers ---------- */
views.market=()=>`<div class="bar"><input id="q" placeholder="Search items..." oninput="drawItems()"><select
    id="fc" onchange="drawItems()"><option value="">All
    categories</option>${db.cats.map(c=>`<option>${c}</option>`).join("")}</select>
  <select id="fp" onchange="drawItems()"><option value="">Any price</option><option value="1000">Under
      ₹1,000</option><option value="5000">Under ₹5,000</option><option value="25000">Under
      ₹25,000</option></select></div><div class="grid" id="items">${itemCards()}</div>`;
function itemCards(){
  const q=($("q")?$("q").value:"").toLowerCase(),c=$("fc")?$("fc").value:"",p=$("fp")?$("fp").value:"";
  const l=db.items.filter(i=>!i.sold&&(!c||i.cat===c)&&(!p||i.price<=+p)&&i.title.toLowerCase().includes(q));
  return l.map(i=>`<div class="card"><b>${i.title}</b><br><span class="tag">${i.cat}</span><br><span
      class="price">${inr(i.price)}</span><br><span class="small">${i.desc} | Seller: ${i.seller}</span><br><br>
    <button class="btn" onclick="buy(${i.id})">Buy Now</button><br><br><div class="bar"><input id="of${i.id}"
        type="number" placeholder="Your offer ₹"><button class="btn blue"
        onclick="offer(${i.id})">Negotiate</button></div></div>`).join("")||"<p>No items found.</p>";
}
function drawItems(){ $("items").innerHTML=itemCards(); }
function needLogin(){ if(role!=="user"){ note("Please login as a user first.",true); go("login"); return true; }
     return false; }
function buy(id,price){
  if(needLogin()) return; const i=db.items.find(x=>x.id===id);
  if(i.seller===me.user) return note("You cannot buy your own item.",true);
  i.sold=true; db.orders.push({id:db.nid.order++,item:i.title,buyer:me.user,seller:i.seller,price:price||i.price
      ,status:"Pending"}); persist(); note("Order placed for "+i.title+"."); go("tx");
}
function offer(id){
  if(needLogin()) return; const v=+$("of"+id).value, i=db.items.find(x=>x.id===id);
  if(!v||v<=0) return note("Enter a valid offer.",true);

      db.offers.push({id:db.nid.offer++,item:id,title:i.title,buyer:me.user,seller:i.seller,amt:v,status:"Pendin
      g"}); persist(); note("Offer of "+inr(v)+" sent to "+i.seller+".");
}

/* ---------- Seller: create listing ---------- */
views.sell=()=>{ if(role!=="user") return "<div class='card'>Please login to create a listing.</div>";
  return `<div class="card"><h3>Create Listing</h3><div class="bar"><input id="st" placeholder="Title"><select
      id="sc">${db.cats.map(c=>`<option>${c}</option>`).join("")}</select><input id="sp" type="number"
      placeholder="Price ₹"></div>
  <textarea id="sd" rows="2" placeholder="Description / condition"></textarea><br><br><button class="btn"
      onclick="addListing()">Post Listing</button></div>`; };
function addListing(){
  if(!$("st").value.trim()||!$("sp").value) return note("Title and price are required.",true);
  db.items.push({id:db.nid.item++,title:$("st").value,cat:$("sc").value,price:+$("sp").value,seller:me.user,desc
      :$("sd").value||"-",sold:false}); persist(); note("Listing posted."); go("market");
}

/* ---------- Transactions: offers, orders, feedback ---------- */
views.tx=()=>{ if(role!=="user") return "<div class='card'>Please login.</div>";
  const rec=db.offers.filter(o=>o.seller===me.user), sent=db.offers.filter(o=>o.buyer===me.user),
      ord=db.orders.filter(o=>o.buyer===me.user||o.seller===me.user);
  return `<div class="card"><h3>Offers Received</h3>${rec.length?`<table><tr><th>Item</th><th>Buyer</th><th>Offe
      r</th><th>Status</th><th></th></tr>${rec.map(o=>`<tr><td>${o.title}</td><td>${o.buyer}</td><td>${inr(o.amt
      )}</td><td>${o.status}</td>
    <td>${o.status==="Pending"?`<button class="btn" onclick="decide(${o.id},true)">Accept</button> <button
        class="btn red"
        onclick="decide(${o.id},false)">Reject</button>`:""}</td></tr>`).join("")}</table>`:"<span
        class='small'>None.</span>"}</div>
  <div class="card"><h3>Offers Sent</h3>${sent.length?`<table><tr><th>Item</th><th>My offer</th><th>Status</th><
      /tr>${sent.map(o=>`<tr><td>${o.title}</td><td>${inr(o.amt)}</td><td>${o.status}${o.status==="Accepted"?`
      <button class="btn" onclick="payOffer(${o.id})">Buy at
      ${inr(o.amt)}</button>`:""}</td></tr>`).join("")}</table>`:"<span class='small'>None.</span>"}</div>
  <div class="card"><h3>My Orders (Purchases & Sales)</h3>${ord.length?`<table><tr><th>#</th><th>Item</th><th>Bu
      yer</th><th>Seller</th><th>Price</th><th>Logistics</th><th>Feedback</th></tr>${ord.map(o=>`<tr><td>${o.id}
      </td><td>${o.item}</td><td>${o.buyer}</td><td>${o.seller}</td><td>${inr(o.price)}</td><td><b>${o.status}</
      b></td>
    <td>${o.buyer===me.user&&o.status==="Received"&&!db.fb.some(f=>f.order===o.id)?`<input id="fb${o.id}"
        placeholder="Feedback" style="width:120px"> <button class="btn" onclick="sendFb(${o.id})">Send</button>`
        :(db.fb.find(f=>f.order===o.id)||{}).text||""}</td></tr>`).join("")}</table>`:"<span class='small'>No
        transactions yet.</span>"}</div>`; };
function decide(id,ok){ db.offers.find(o=>o.id===id).status=ok?"Accepted":"Rejected"; persist(); note("Offer
    "+(ok?"accepted":"rejected")+"."); go("tx"); }
function payOffer(id){ const o=db.offers.find(x=>x.id===id); const i=db.items.find(x=>x.id===o.item); if(i.sold)
     return note("Item already sold.",true); buy(i.id,o.amt); }
function sendFb(id){ const t=$("fb"+id).value.trim(); if(!t) return note("Write feedback first.",true);
    db.fb.push({order:id,text:t}); persist(); note("Thanks for your feedback!"); go("tx"); }

/* ---------- ADMIN: user management ---------- */
views.admin_users=()=>`<div class="card"><h3>Create User Account (set login ID & password)</h3><div
    class="bar"><input id="an" placeholder="Full name"><input id="au" placeholder="Username"><input id="ap"
    placeholder="Password"><button class="btn" onclick="aAddUser()">Create</button></div></div>
  <div class="card"><h3>User Management</h3><table><tr><th>ID</th><th>Name</th><th>Username</th><th>Password</th
      ><th></th></tr>${db.users.map(u=>`<tr><td>${u.id}</td>
  <td><input id="un${u.id}" value="${u.name}"></td><td>${u.user}</td><td><input id="up${u.id}"
      value="${u.pass}"></td>
  <td><button class="btn" onclick="aSaveUser(${u.id})">Save</button> <button class="btn red"
      onclick="aDelUser(${u.id})">Delete</button></td></tr>`).join("")}</table></div>`;
```

```
function aAddUser(){ if(!$("an").value||!$("au").value||!$("ap").value) return note("Fill all fields.",true);
    if(db.users.some(u=>u.user===$("au").value)) return note("Username exists.",true);
  db.users.push({id:db.nid.user++,name:$("an").value,user:$("au").value,pass:$("ap").value,email:"-"});
      persist(); note("User account created."); go("admin_users"); }
function aSaveUser(id){ const u=db.users.find(x=>x.id===id); u.name=$("un"+id).value; u.pass=$("up"+id).value;
    persist(); note("User updated."); }
function aDelUser(id){ db.users=db.users.filter(u=>u.id!==id); persist(); note("User deleted.");
    go("admin_users"); }

/* ---------- ADMIN: category management ---------- */
views.admin_cats=()=>`<div class="card"><h3>Category Management</h3><div class="bar"><input id="nc"
    placeholder="New category name"><button class="btn" onclick="aAddCat()">Add Category</button></div>
  <table><tr><th>Category</th><th>Items</th><th></th></tr>${db.cats.map(c=>`<tr><td>${c}</td><td>${db.items.filt
      er(i=>i.cat===c).length}</td><td><button class="btn red"
      onclick="aDelCat('${c}')">Delete</button></td></tr>`).join("")}</table></div>`;
function aAddCat(){ const c=$("nc").value.trim(); if(!c||db.cats.includes(c)) return note("Enter a new category
    name.",true); db.cats.push(c); persist(); note("Category added."); go("admin_cats"); }
function aDelCat(c){ db.cats=db.cats.filter(x=>x!==c); persist(); note("Category deleted."); go("admin_cats"); }

/* ---------- ADMIN: commodity management ---------- */
views.admin_items=()=>`<div class="card"><h3>Commodity Management</h3><table><tr><th>ID</th><th>Title</th><th>Ca
    tegory</th><th>Price</th><th>Seller</th><th>Status</th><th></th></tr>${db.items.map(i=>`<tr><td>${i.id}</td>
  <td><input id="it${i.id}" value="${i.title}"></td><td>${i.cat}</td><td><input id="ip${i.id}" type="number"
      value="${i.price}" style="width:90px"></td><td>${i.seller}</td><td>${i.sold?"Sold":"Available"}</td>
  <td><button class="btn" onclick="aSaveItem(${i.id})">Save</button> <button class="btn red"
      onclick="aDelItem(${i.id})">Delete</button></td></tr>`).join("")}</table></div>`;
function aSaveItem(id){ const i=db.items.find(x=>x.id===id); i.title=$("it"+id).value;
    i.price=+$("ip"+id).value; persist(); note("Commodity updated."); }
function aDelItem(id){ db.items=db.items.filter(i=>i.id!==id); persist(); note("Commodity deleted.");
    go("admin_items"); }

/* ---------- ADMIN: order management (logistics status) ---------- */
views.admin_orders=()=>`<div class="card"><h3>Order Management</h3>${db.orders.length?`<table><tr><th>#</th><th>
    Item</th><th>Buyer</th><th>Seller</th><th>Price</th><th>Logistics Status</th><th></th></tr>${db.orders.map(o
    =>`<tr><td>${o.id}</td><td>${o.item}</td><td>${o.buyer}</td><td>${o.seller}</td><td>${inr(o.price)}</td>
  <td><select onchange="aStatus(${o.id},this.value)">${["Pending","Shipped","Received"].map(s=>`<option
      ${s===o.status?"selected":""}>${s}</option>`).join("")}</select></td>
  <td><button class="btn red" onclick="aDelOrder(${o.id})">Delete
      Order</button></td></tr>`).join("")}</table>`:"No orders yet."}</div>`;
function aStatus(id,s){ db.orders.find(o=>o.id===id).status=s; persist(); note("Order #"+id+" marked "+s+"."); }
function aDelOrder(id){ db.orders=db.orders.filter(o=>o.id!==id); persist(); note("Order deleted.");
    go("admin_orders"); }

go("market");
</script>
</body>
</html>
```

## OUTPUT

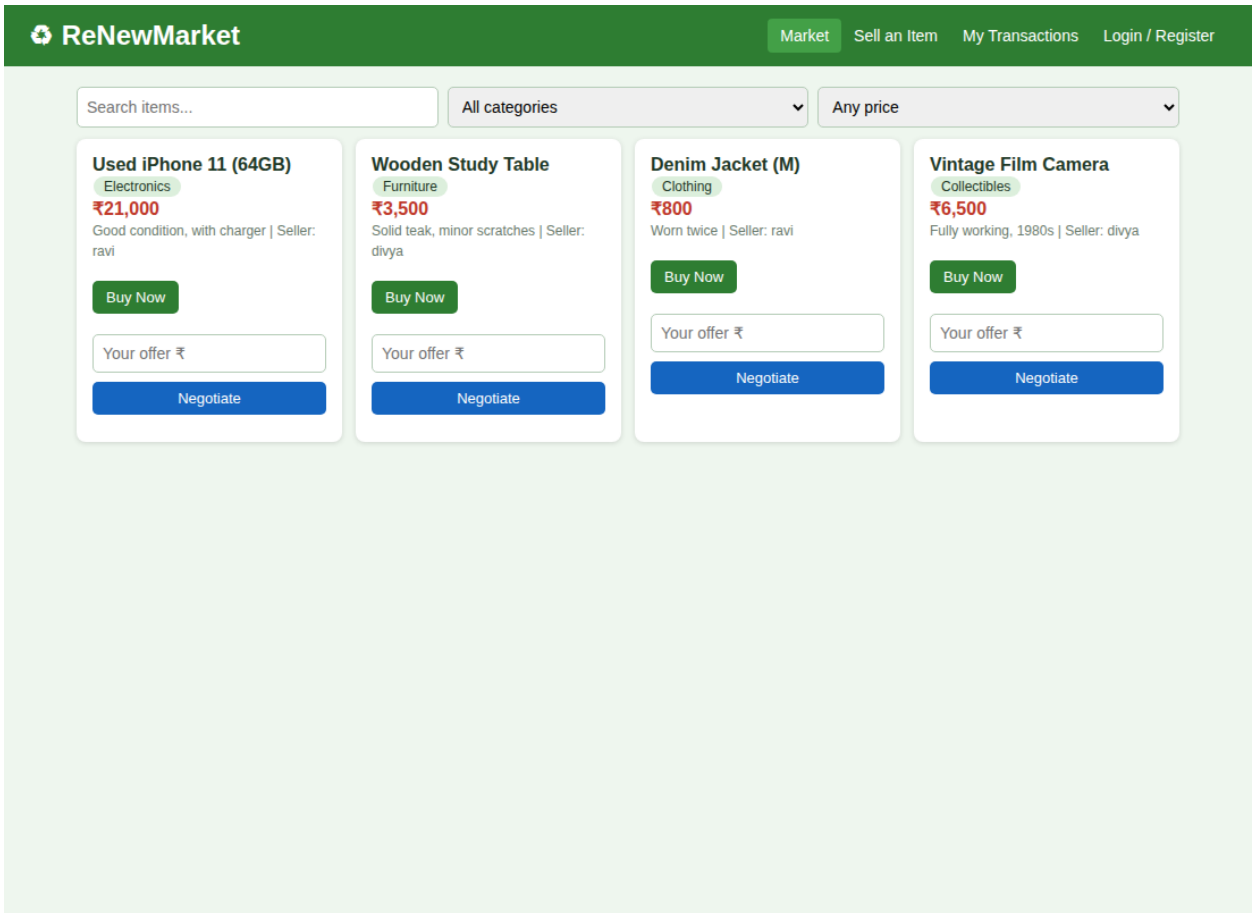



*Fig 14.1: Home page: product catalogue with search box, category filter and price filter*

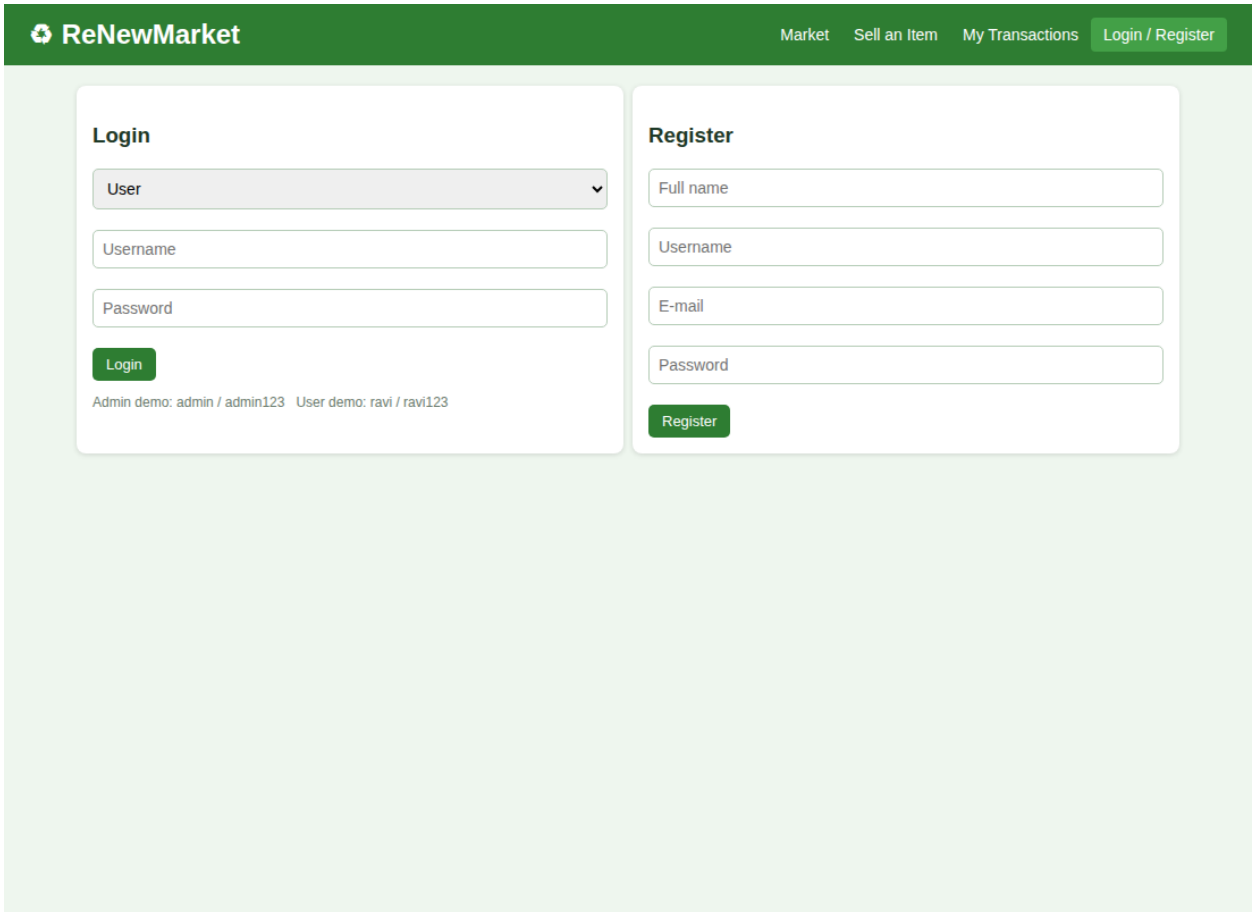



*Fig 14.2: Login / Register page for users and administrator*

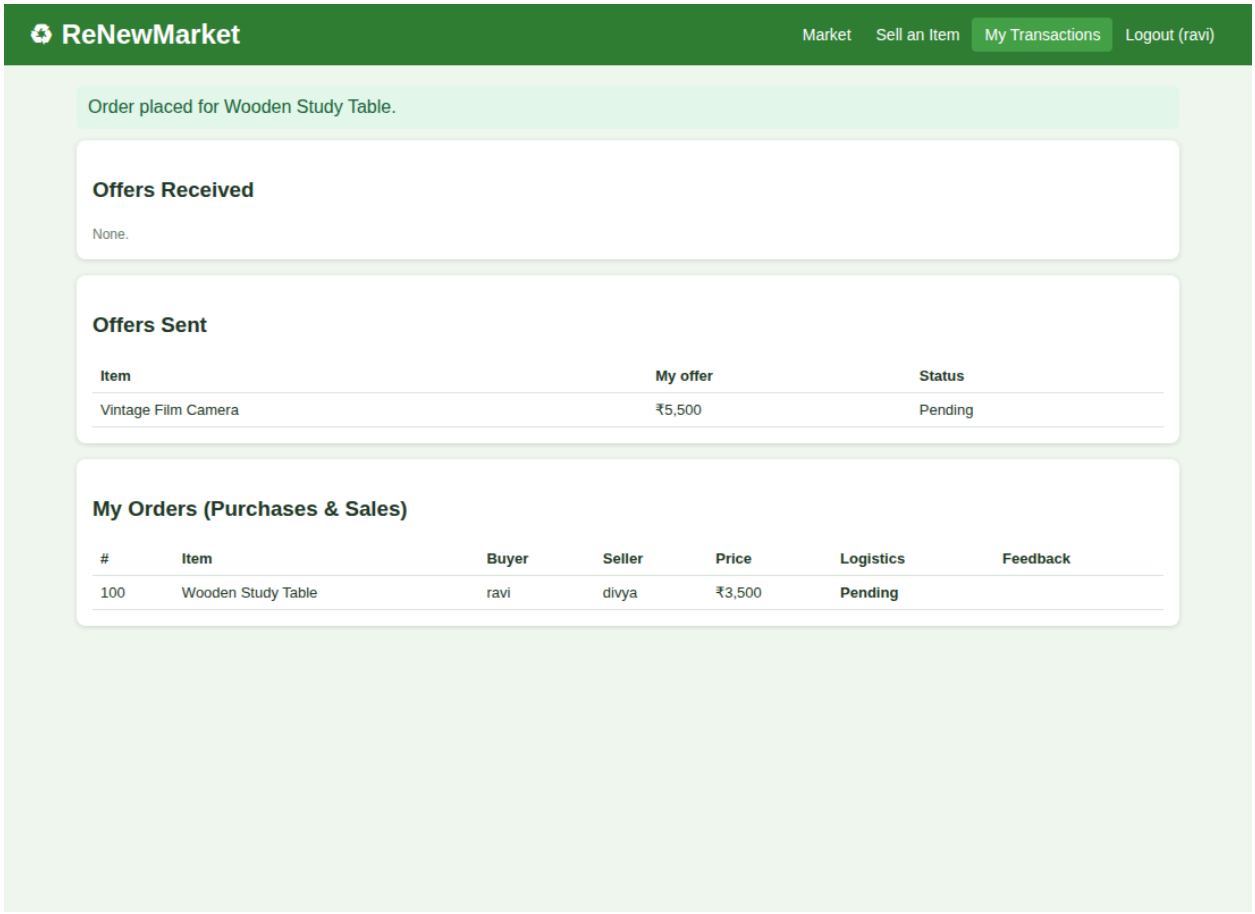



*Fig 14.3: User "ravi" sends a price offer for the Vintage Film Camera and buys the Wooden Study Table (My Transactions)*

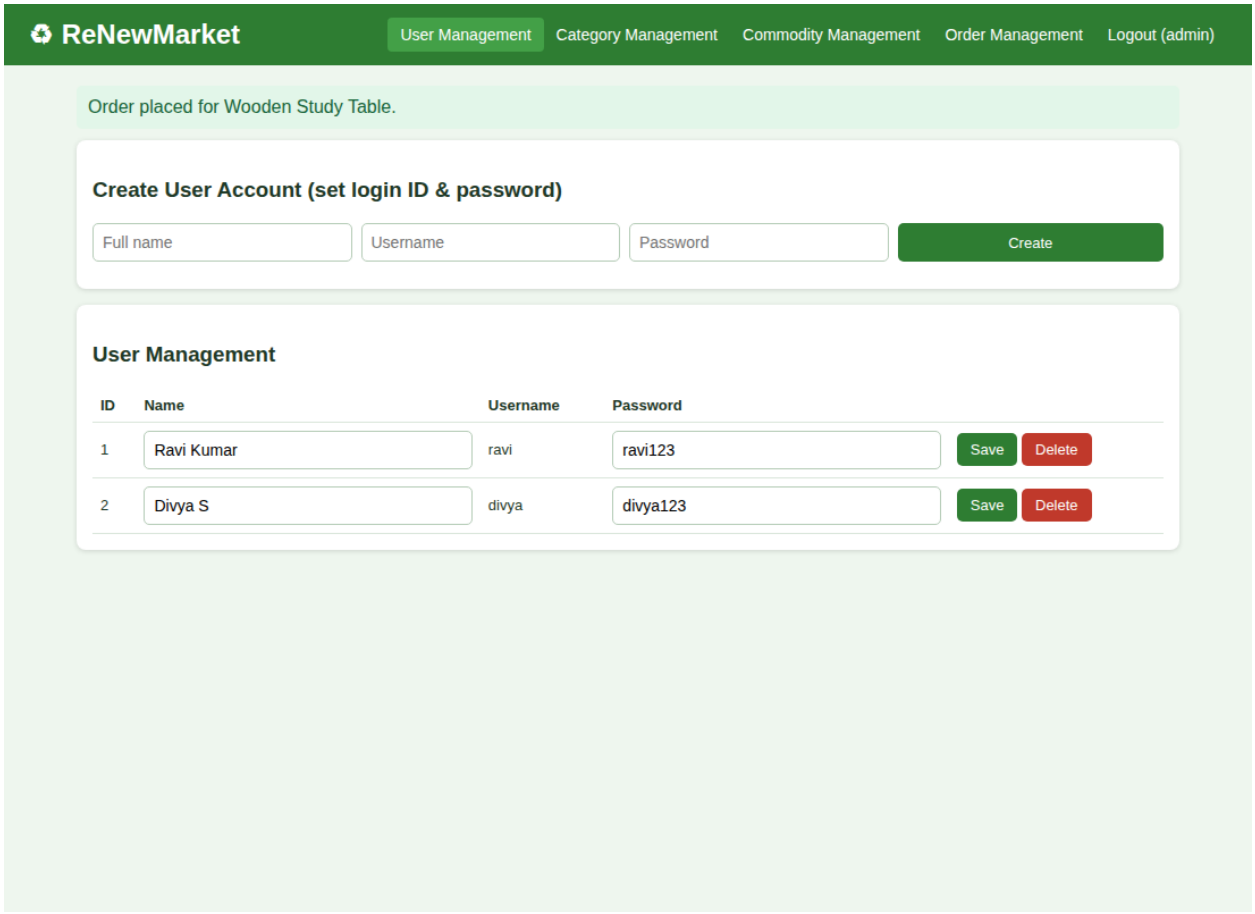



*Fig 14.4: Administrator login: User Management (create, edit and delete user accounts)*

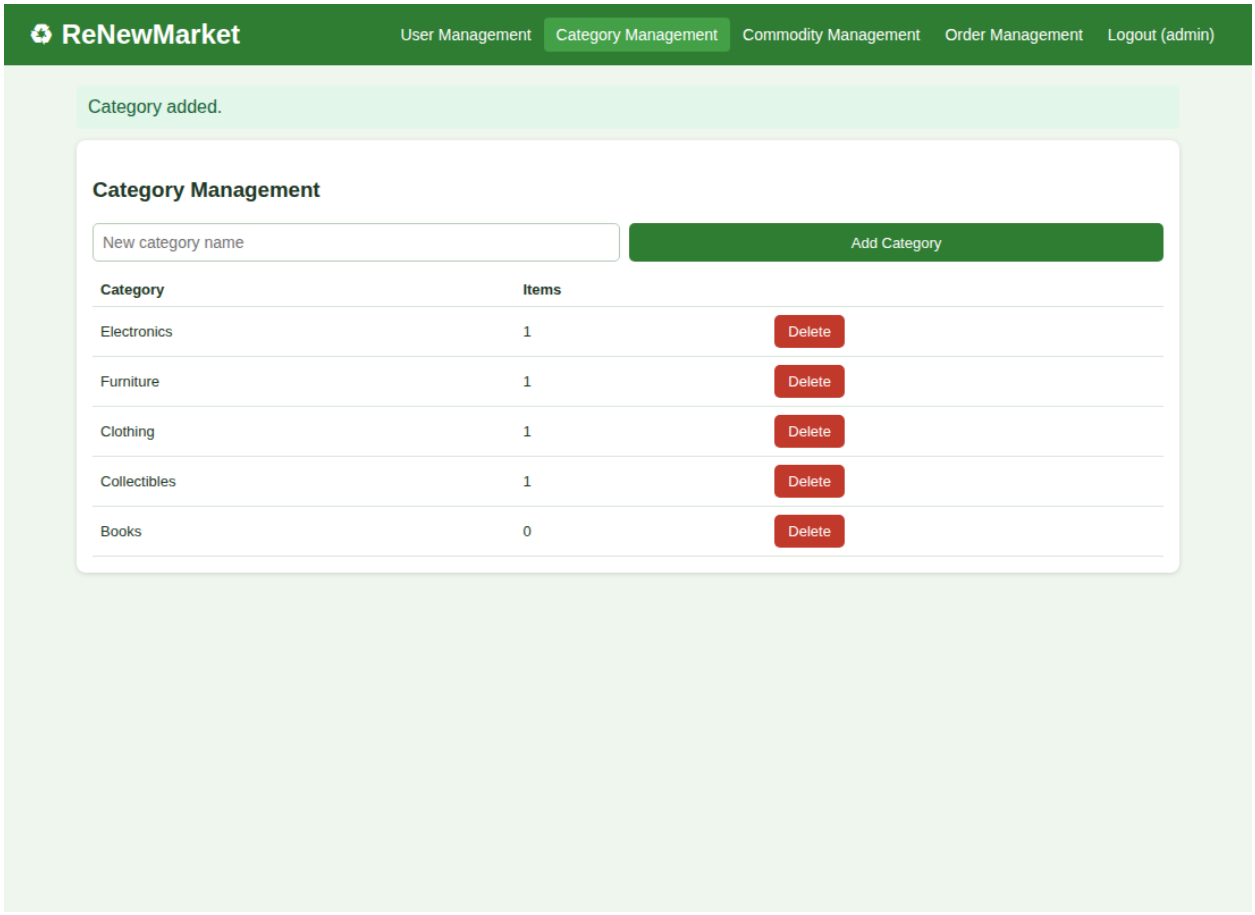



*Fig 14.5: Category Management: new category "Books" added*

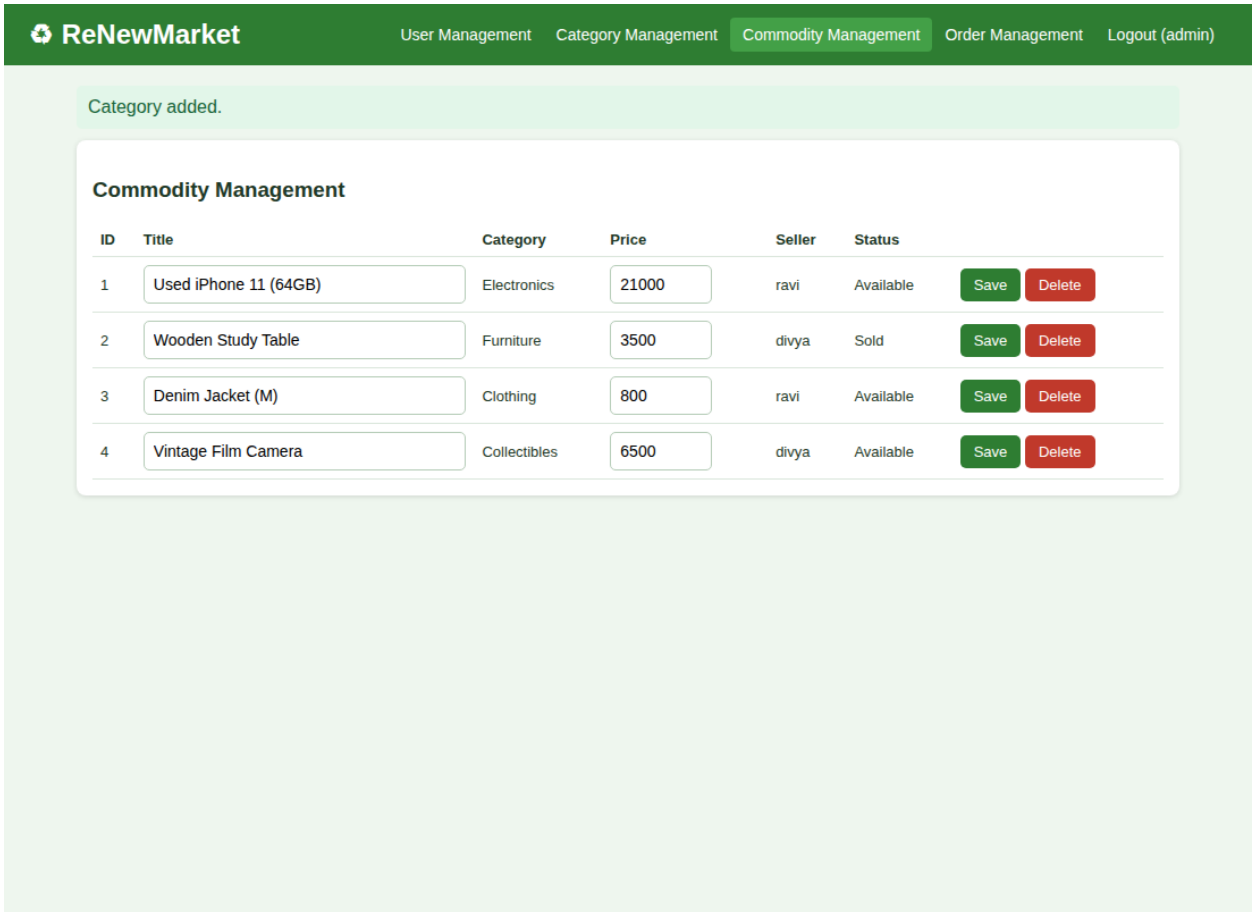



*Fig 14.6: Commodity Management: edit price / delete items (the sold item is shown as Sold)*

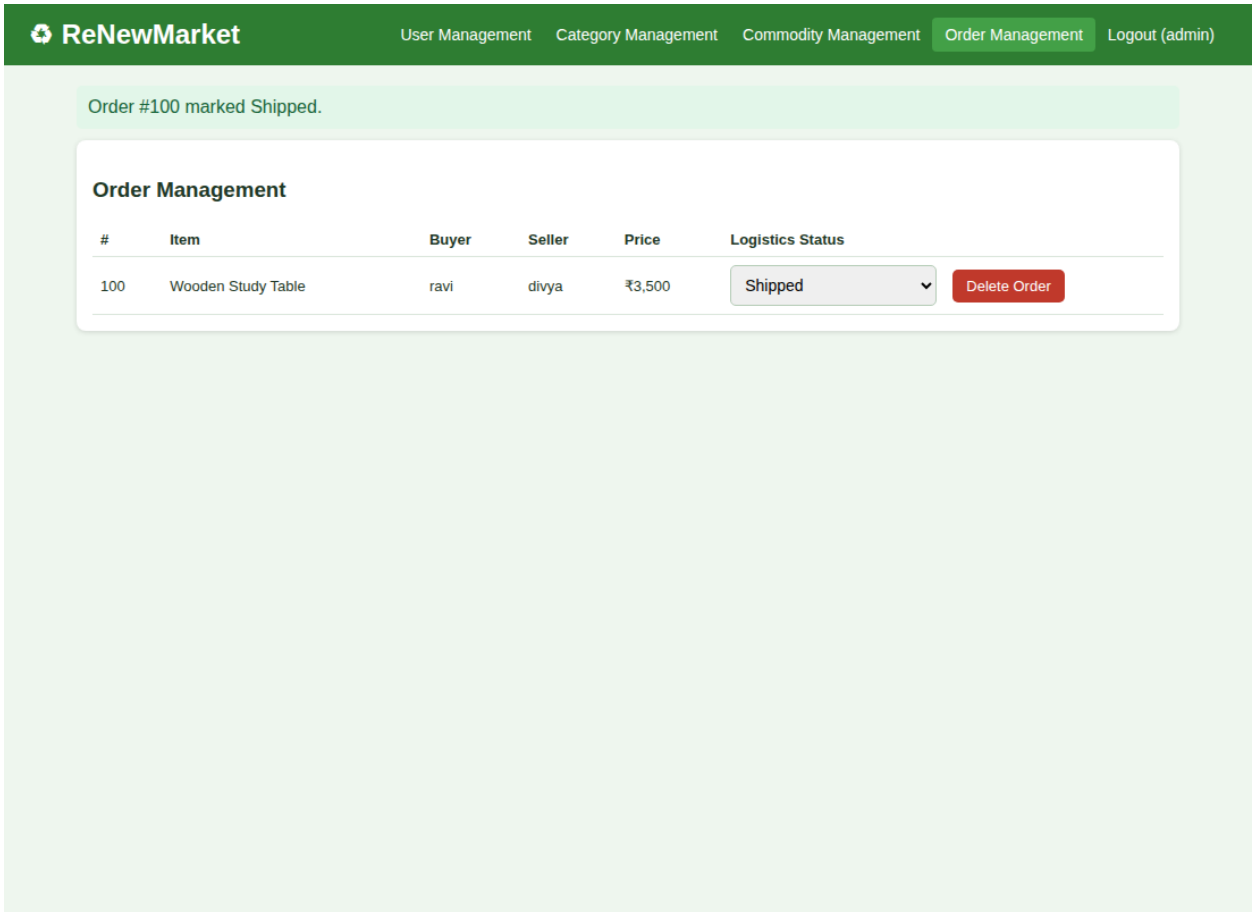



*Fig 14.7: Order Management: logistics status of order #100 changed to Shipped*

## RESULT

Thus the ReNewMarket second-hand marketplace was designed and implemented using HTML, CSS and JavaScript. Users could register, post items, search, negotiate and buy, and the administrator could manage users, categories, commodities and order logistics status. The output was verified successfully.